МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
<БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ>

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

# Отчет по лабораторной работе №2

Специальность АС-67

Выполнил
И. А. Якимович,
студент группы АС-67

Проверил
А. А. Крощенко,
ст. преп. кафедры ИИТ,
«___> _____2026 г.

Брест 2026

**Цель работы**: изучить применение линейной и логистической регрессии для решения практических задач. Научиться обучать модели, оценивать их качество с помощью соответствующих метрик и интерпретировать результаты.

**Вариант 1**

**Задание.** выполнить задания по варианту (регрессия и классификация), построить все требуемые визуализации и рассчитать метрики.

- **Регрессия (Прогнозирование стоимости жилья в Калифорнии)**
  1. California Housing
  2. Предсказать медианную стоимость дома (median_house_value)
  3. **Задания:**
     - загрузите данные и разделите их на обучающую и тестовую выборки;
     - обучите модель **линейной регрессии** на обучающих данных;
     - сделайте предсказания для тестовой выборки;
     - оцените качество модели, рассчитав метрики **MSE** (Mean Squared Error) и **$R^2$** (Coefficient of Determination);
     - визуализируйте результат: постройте диаграмму рассеяния для признака median_income (медианный доход) и целевой переменной, нанеся на неё линию регрессии.
- **Классификация (Прогнозирование выживаемости на "Титанике")**

  1. Titanic
  2. Предсказать, выжил ли пассажир (Survived)
  3. **Задания:**
     - загрузите и предварительно обработайте данные (заполните пропуски, преобразуйте категории в числа);
     - обучите модель **логистической регрессии**;
     - оцените качество модели, рассчитав **Accuracy**, **Precision** и **Recall**;

- постройте и проанализируйте **матрицу ошибок** (confusion matrix).

Выполнение:

## Код программы

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from sklearn.datasets import fetch_california_housing
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    mean_squared_error,
    precision_score,
    recall_score,
    r2_score,
)
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

print("   Запуск задачи регрессии (California Housing)")

housing = fetch_california_housing(as_frame=True)

df_california = housing.frame.copy()
```

```
df_california = df_california.rename(columns={
    "MedInc": "median_income",
    "MedHouseVal": "median_house_value"
})

df_california["median_house_value"] *= 100000

X_reg = df_california.drop("median_house_value", axis=1)
y_reg = df_california["median_house_value"]

X_train_r, X_test_r, y_train_r, y_test_r = train_test_split(
    X_reg, y_reg, test_size=0.2, random_state=42
)

lin_reg = LinearRegression()
lin_reg.fit(X_train_r, y_train_r)

y_pred_r = lin_reg.predict(X_test_r)

mse = mean_squared_error(y_test_r, y_pred_r)
r2 = r2_score(y_test_r, y_pred_r)

print("Количество записей:", len(df_california))
print("Количество признаков:", X_reg.shape[1])
print(f"MSE: {mse:.4f}")
print(f"R^2: {r2:.4f}")

df_california.to_csv("california_housing.csv", index=False)

plt.figure(figsize=(8, 6))

plt.scatter(
    df_california["median_income"],
    df_california["median_house_value"],
    alpha=0.5,
    color="blue",
    label="Данные"
)

m, b = np.polyfit(
    df_california["median_income"],
    df_california["median_house_value"],
    1
)

x_line = np.linspace(
    df_california["median_income"].min(),
    df_california["median_income"].max(),
    100
)

plt.plot(
    x_line,
    m * x_line + b,
    color="red",
    label="Линия регрессии"
)

plt.title("Зависимость стоимости жилья от медианного дохода")
plt.xlabel("Median Income")
plt.ylabel("Median House Value ($)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

print("\n   Запуск задачи классификации (Titanic)")
```

```
df_titanic = pd.read_csv("Titanic-Dataset.csv")

df_titanic.columns = df_titanic.columns.str.strip().str.lower()

features = [
    "pclass",
    "sex",
    "age",
    "sibsp",
    "parch",
    "fare",
    "embarked"
]

X_clf = df_titanic[features].copy()
y_clf = df_titanic["survived"].astype(int)

print("Количество записей:", len(df_titanic))
print("Количество признаков:", X_clf.shape[1])
print("Не выжили:", (y_clf == 0).sum())
print("Выжили:", (y_clf == 1).sum())

X_train_c, X_test_c, y_train_c, y_test_c = train_test_split(
    X_clf,
    y_clf,
    test_size=0.2,
    random_state=42,
    stratify=y_clf
)

numeric_features = [
    "pclass",
    "age",
    "sibsp",
    "parch",
    "fare"
]

categorical_features = [
    "sex",
    "embarked"
]

for column in numeric_features:
    median = X_train_c[column].median()

    X_train_c[column] = X_train_c[column].fillna(median)
    X_test_c[column] = X_test_c[column].fillna(median)

for column in categorical_features:
    mode = X_train_c[column].mode()[0]

    X_train_c[column] = X_train_c[column].fillna(mode)
    X_test_c[column] = X_test_c[column].fillna(mode)

X_train_c = pd.get_dummies(
    X_train_c,
    columns=categorical_features,
    drop_first=True,
    dtype=int
)

X_test_c = pd.get_dummies(
    X_test_c,
    columns=categorical_features,
    drop_first=True,
    dtype=int
```

```
)

X_test_c = X_test_c.reindex(
    columns=X_train_c.columns,
    fill_value=0
)

scaler = StandardScaler()

X_train_c = scaler.fit_transform(X_train_c)
X_test_c = scaler.transform(X_test_c)

log_reg = LogisticRegression(
    random_state=42,
    max_iter=1000
)

log_reg.fit(X_train_c, y_train_c)

y_pred_c = log_reg.predict(X_test_c)

accuracy = accuracy_score(y_test_c, y_pred_c)
precision = precision_score(y_test_c, y_pred_c, zero_division=0)
recall = recall_score(y_test_c, y_pred_c, zero_division=0)

cm = confusion_matrix(y_test_c, y_pred_c)

print("\nРЕЗУЛЬТАТЫ КЛАССИФИКАЦИИ")
print(f"Accuracy:  {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall:    {recall:.4f}")

print("\nМАТРИЦА ОШИБОК")
print(cm)

tn, fp, fn, tp = cm.ravel()

print("\nАНАЛИЗ МАТРИЦЫ ОШИБОК")
print(f"Истинно отрицательные (TN): {tn}")
print(f"Ложноположительные (FP): {fp}")
print(f"Ложноотрицательные (FN): {fn}")
print(f"Истинно положительные (TP): {tp}")
print(f"Сумма элементов матрицы: {cm.sum()}")

plt.figure(figsize=(6, 5))

plt.imshow(
    cm,
    interpolation="nearest",
    cmap=plt.cm.Blues
)

plt.title("Матрица ошибок — Titanic")
plt.colorbar()

tick_marks = np.arange(2)

plt.xticks(tick_marks, ["Не выжил", "Выжил"])
plt.yticks(tick_marks, ["Не выжил", "Выжил"])

thresh = cm.max() / 2.0

for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        plt.text(
            j,
            i,
            format(cm[i, j], "d"),
```

```
            horizontalalignment="center",
            color="white" if cm[i, j] > thresh else "black"
        )

plt.xlabel("Предсказанные значения")
plt.ylabel("Истинные значения")
plt.tight_layout()
plt.show()
```

## Результат:

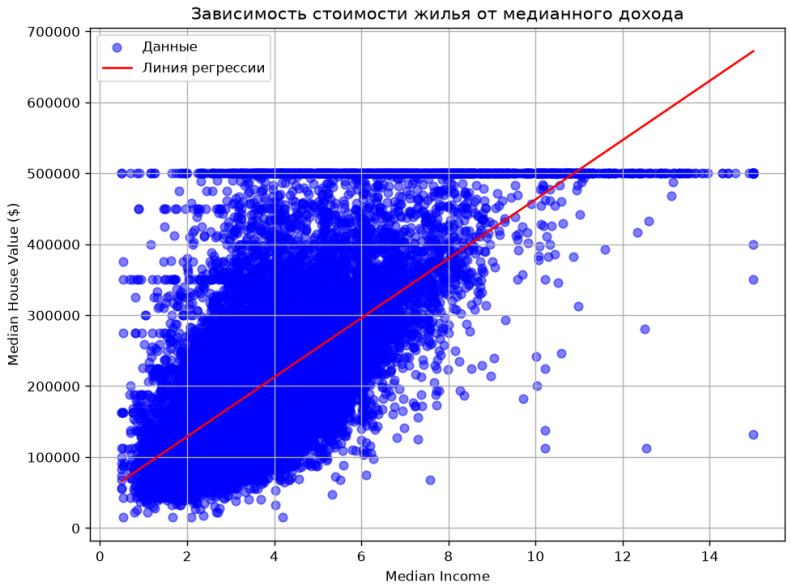
Запуск задачи регрессии (California Housing)
Количество записей: 20640
Количество признаков: 8
MSE: 5558915986.9524
R^2: 0.5758

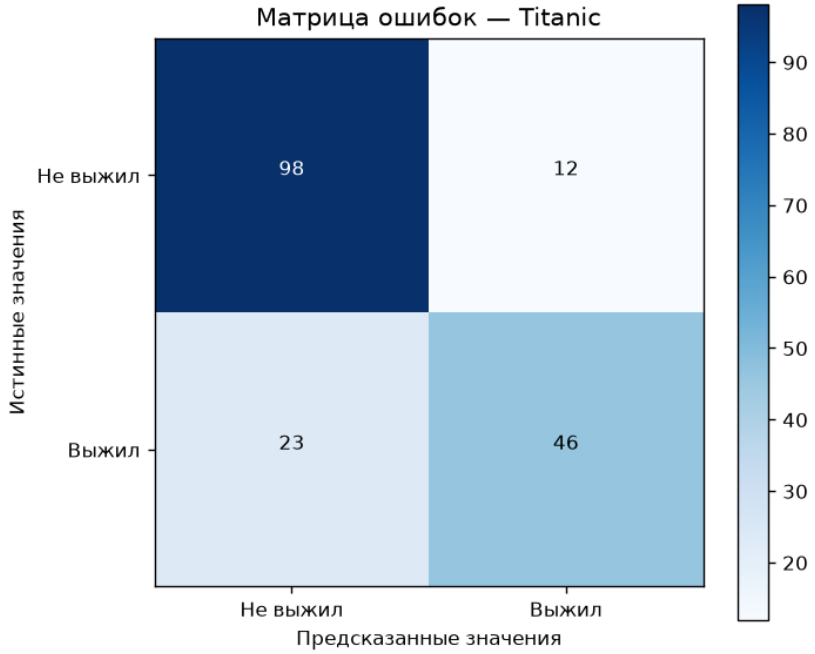
Запуск задачи классификации (Titanic)
Количество записей: 891
Количество признаков: 7
Не выжили: 549
Выжили: 342

РЕЗУЛЬТАТЫ КЛАССИФИКАЦИИ
Accuracy:  0.8045
Precision: 0.7931
Recall:  0.6667

МАТРИЦА ОШИБОК
[[98 12]
 [23 46]]

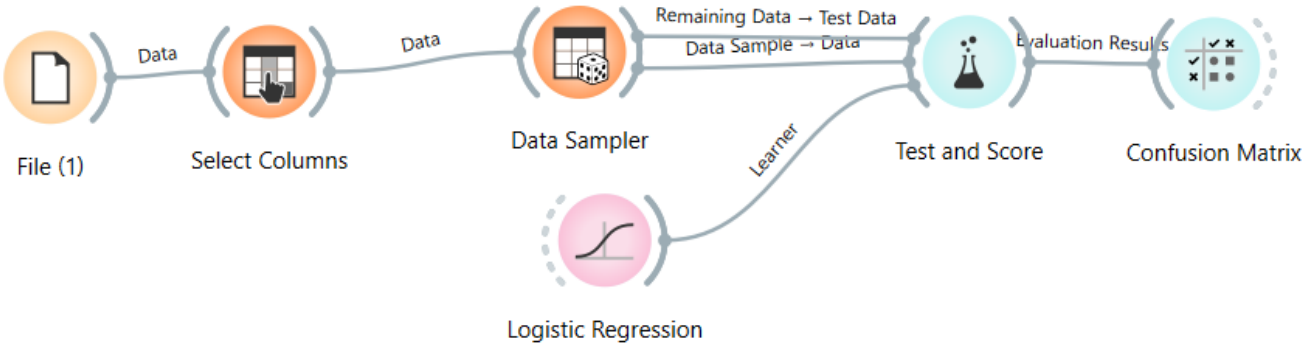
АНАЛИЗ МАТРИЦЫ ОШИБОК
Истинно отрицательные (TN): 98
Ложноположительные (FP): 12
Ложноотрицательные (FN): 23
Истинно положительные (TP): 46
Сумма элементов матрицы: 179

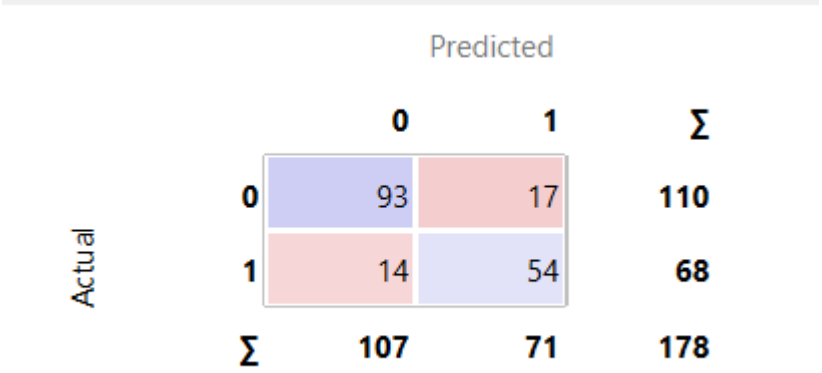


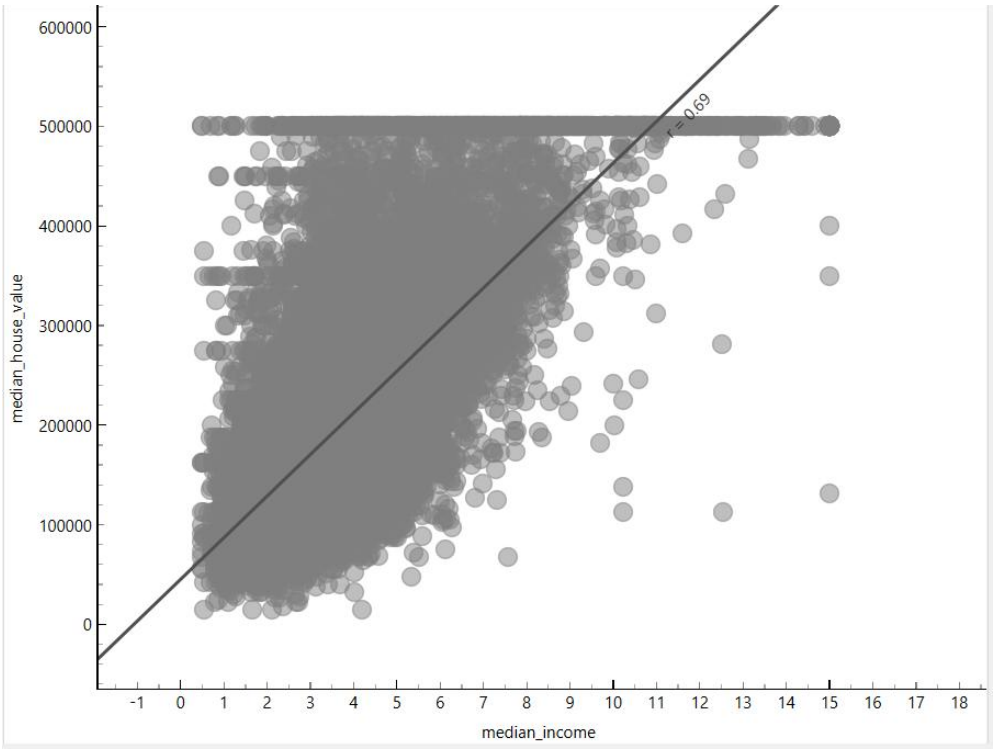



## Результаты в Orange:






**Вывод:** в ходе выполнения лабораторной работы были успешно освоены методы линейной и логистической регрессии для решения практических задач. Были обучены модели для задач регрессии и классификации, рассчитаны ключевые метрики их качества и проанализированы полученные результаты.